GPU and CUDA

Prof. V. Mele

Parallel High Performance Computing

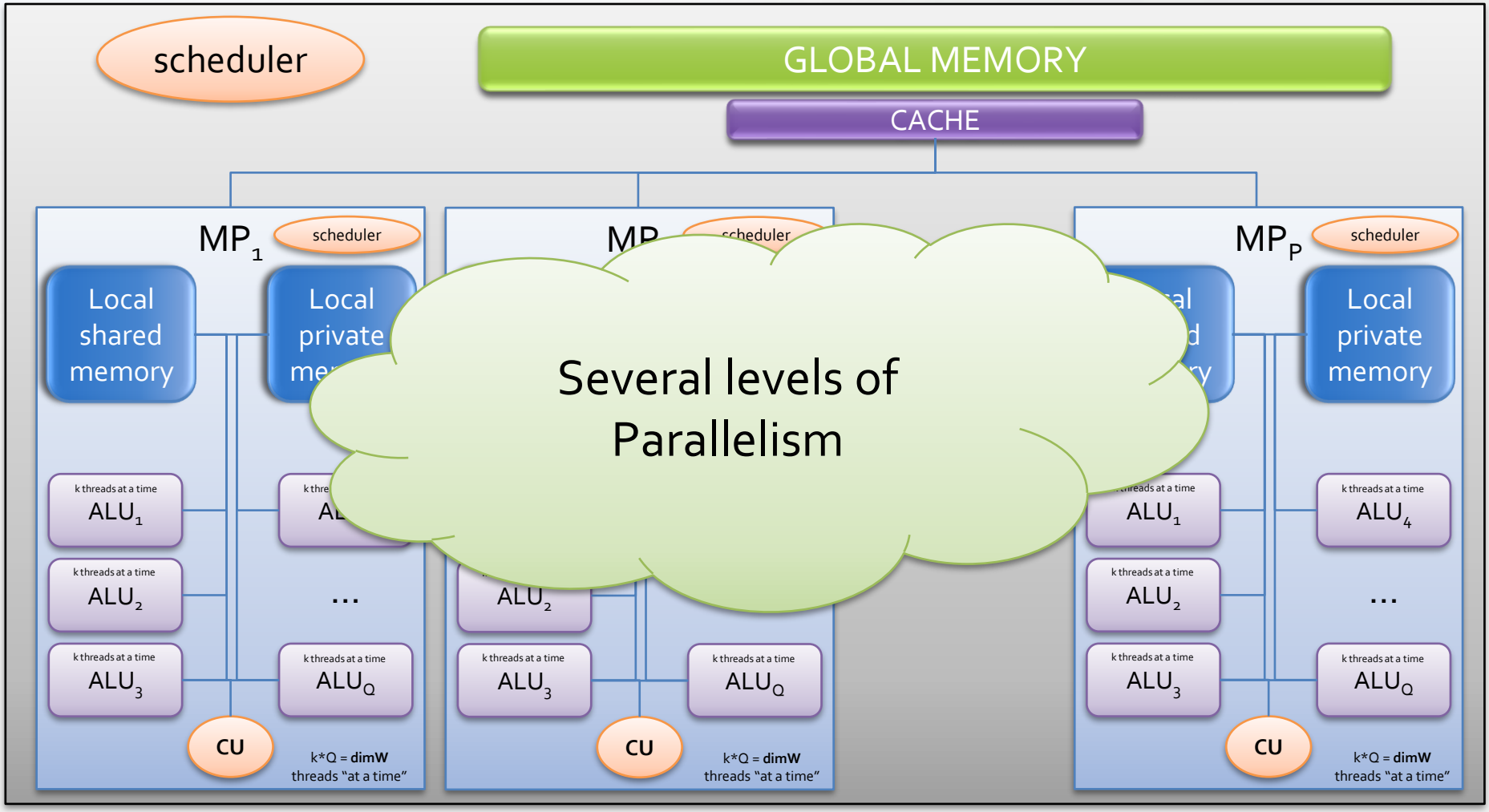
University of Naples Federico II

a.a. 2025/26

GPU

G80

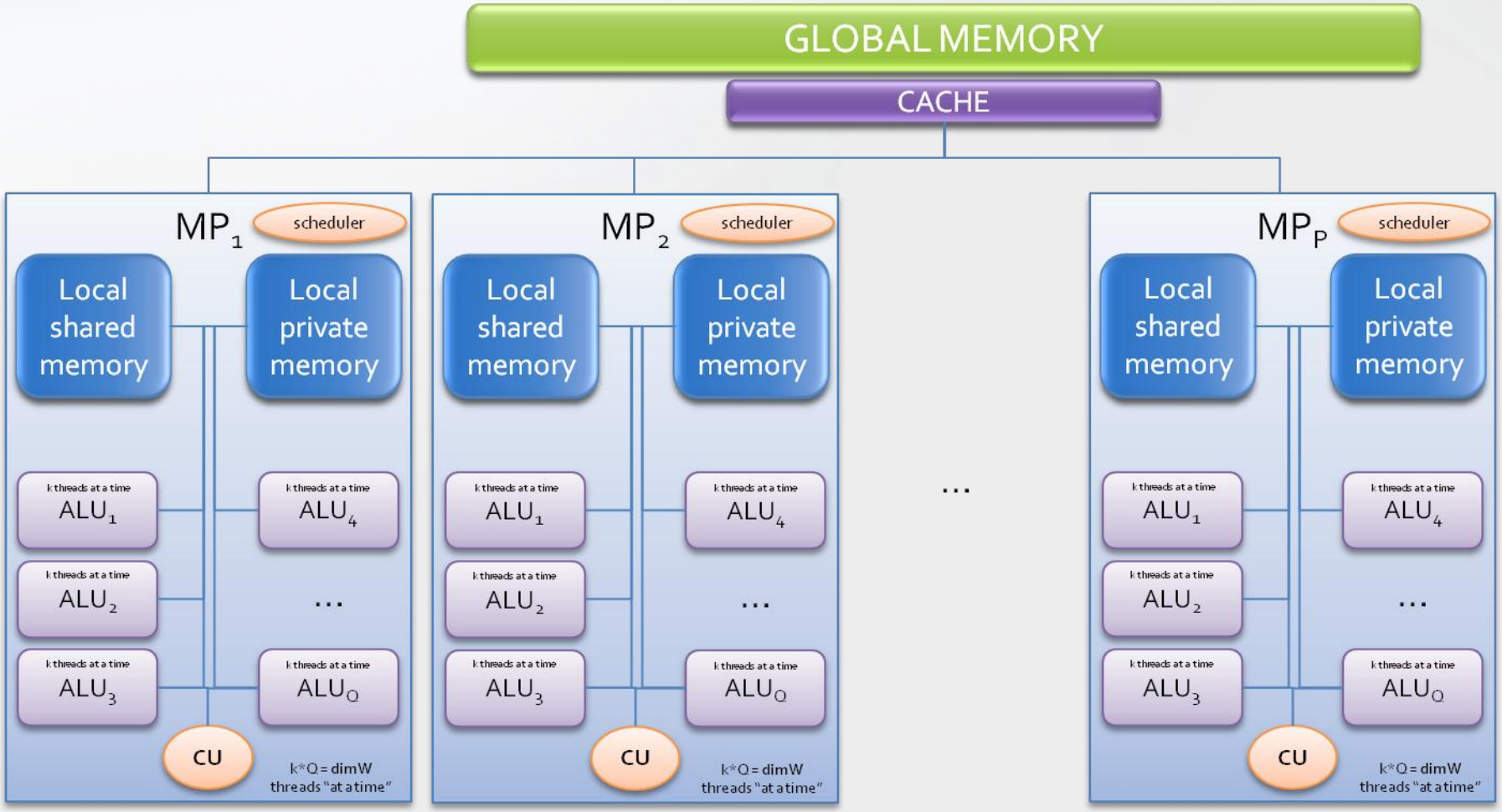
GPU Architecture: Several levels of Parallelism



GPU

G80

GPU Architecture: Several levels of Parallelism



Evolution of NVIDIA GPUs

Architectures

G80

Tesla (2006-2009)

- First CUDA generation.
- Support for FP32 and FP64, but with limited computing power compared to today's standards.

Evolution of NVIDIA GPUs

Architectures

Tesla (2006-2009)

- First CUDA generation.
- Support for FP32 and FP64, but with limited computing power compared to today's standards.

G80

- released in **2006,** was the first GPU based on the **Tesla architecture**
- first chip to support **CUDA**
- **128 stream processors (SPs)** organized into **Streaming Multiprocessors (SMs)**

Evolution of NVIDIA GPUs

Architectures

Tesla (2006-2009)

- First CUDA generation.
- Support for FP32 and FP64, but with limited computing power compared to today's standards.
- Max performance FP32: ~1 TFLOP (1000 GFLOPS) with Tesla C1060.

based on the **GT200** architecture, an evolution of **G80**

G80

- released in **2006**, was the first GPU based on the **Tesla architecture**
- first chip to support **CUDA**
- **128 stream processors (SPs)** organized into **Streaming Multiprocessors (SMs)**

Evolution of NVIDIA GPUs

Architectures

Tesla (2006-2009)

- First CUDA generation.
- Support for FP32 and FP64, but with limited computing power compared to today's standards.
- Max performance FP32: ~1 TFLOP (1000 GFLOPS) with Tesla C1060.

Fermi (2010-2011)

- New architecture with L1/L2 cache.
- FP64 performance doubled compared to Tesla.
- Max performance FP32: ~1.5 TFLOPS with Tesla C2050.

Evolution of NVIDIA GPUs

Architectures

Tesla (2006-2009)

- First CUDA generation.
- Support for FP32 and FP64, but with limited computing power compared to today's standards.
- Max performance FP32: ~1 TFLOP (1000 GFLOPS) with Tesla C1060.

Fermi (2010-2011)

- New architecture with L1/L2 cache.
- FP64 performance doubled compared to Tesla.
- Max performance FP32: ~1.5 TFLOPS with Tesla C2050.

Kepler (2012-2013)

- Increased CUDA Core count and SMs.
- Transition to 28 nm → higher energy efficiency.
- Max performance FP32: ~3.5 TFLOPS with Tesla K20.

Evolution of NVIDIA GPUs

Architectures

Maxwell (2014-2015)

- Optimization for energy efficiency.
- Focused on gaming performance, weaker FP64 capabilities
- Max performance FP32: ~4.6 TFLOPS with GTX 980.

Evolution of NVIDIA GPUs

Architectures

Maxwell (2014-2015)

- Optimization for energy efficiency.
- Focused on gaming performance, weaker FP64 capabilities
- Max performance FP32: ~4.6 TFLOPS with GTX 980.

Pascal (2016)

- Adoption of HBM2 (High Bandwidth Memory).
- Introduction of NVLink for GPU interconnect.
- Max performance FP32: ~10.6 TFLOPS with Tesla P100.

Evolution of NVIDIA GPUs

Architectures

Maxwell (2014-2015)

- Optimization for energy efficiency.
- Focused on gaming performance, weaker FP64 capabilities
- Max performance FP32: ~4.6 TFLOPS with GTX 980.

Pascal (2016)

- Adoption of HBM2 (High Bandwidth Memory).
- Introduction of NVLink for GPU interconnect.
- Max performance FP32: ~10.6 TFLOPS with Tesla P100.

NVLink technology

NVLink is NVIDIA's **high-speed interconnect technology** designed to let GPUs, and in some systems CPUs and GPUs, communicate with each other much faster and more efficiently than with standard interconnects such as PCIe. It is used when moving data between accelerators becomes a bottleneck.

When the system contains many GPUs, NVLink is often used together with **NVSwitch**.

Evolution of NVIDIA GPUs

Architectures

Maxwell (2014-2015)

- Optimization for energy efficiency.
- Focused on gaming performance, weaker FP64 capabilities
- Max performance FP32: ~4.6 TFLOPS with GTX 980.

Pascal (2016)

- Adoption of HBM2 (High Bandwidth Memory).
- Introduction of NVLink for GPU interconnect.
- Max performance FP32: ~10.6 TFLOPS with Tesla P100.

Volta (2017-2019)

- Tensor Cores introduced for AI/Deep Learning acceleration.
- Increased FP64 support for HPC.
- Max performance FP32: ~14.9 TFLOPS with Tesla V100.

Evolution of NVIDIA GPUs

Architectures

Maxwell (2014-2015)

- Optimization for energy efficiency.
- Focused on gaming performance, weaker FP64 capabilities
- Max performance FP32: ~4.6 TFLOPS with GTX

Pascal (2016)

- Adoption of HBM2 (High Bandwidth Memory)
- Introduction of NVLink for GPU interconnect.
- Max performance FP32: ~10.6 TFLOPS with T

Volta (2017-2019)

- Tensor Cores introduced for AI/Deep Learning
- Increased FP64 support for HPC.
- Max performance FP32: ~14.9 TFLOPS with T

TENSOR CORES

Tensor Cores are dedicated hardware units for **high-throughput mixed-precision matrix operations**, primarily Dense linear algebra operations dominate modern deep learning and many HPC applications.

- The key idea is that many important algorithms reduce to repeated **matrix multiply-and-accumulate operations**.
- Tensor Cores are specialized unit for **matrix math** and execute these operations in a highly optimized way, so they can deliver much higher throughput than general-purpose cores for the same type of work.
- Multidimensional arrays are often called **tensors**. Since neural network computation is largely tensor algebra, NVIDIA named these units **Tensor Cores**.

Evolution of NVIDIA GPUs

Architectures

Maxwell (2014-2015)

- Optimization for energy efficiency.
- Focused on gaming performance, weaker FP64 capabilities
- Max performance FP32: ~4.6 TFLOPS with GTX 980.

Pascal (2016)

- Adoption of HBM2 (High Bandwidth Memory).
- Introduction of NVLink for GPU interconnect.
- Max performance FP32: ~10.6 TFLOPS with Tesla P100.

Volta (2017-2019)

- Tensor Cores introduced for AI/Deep Learning acceleration.
- Increased FP64 support for HPC.
- Max performance FP32: ~14.9 TFLOPS with Tesla V100.

Turing architecture for gaming

Evolution of NVIDIA GPUs

Architectures

Ampere (2020-2021)

- Significant AI performance increase with 3rd generation Tensor Cores.
- FP64 performance doubled compared to Volta.
- Max performance FP32: ~19.5 TFLOPS with A100.

Evolution of NVIDIA GPUs

Architectures

Ampere (2020-2021)

- Significant AI performance increase with 3rd generation Tensor Cores.
- FP64 performance doubled compared to Volta.
- Max performance FP32: ~19.5 TFLOPS with A100.

(Grace) **Hopper (2022-2023)**

- Transition to 4 nm manufacturing process (80 billion transistors)
  - Better energy efficiency.
- Support for FP8 for AI → 60 PetaFLOPS on H100!
- Max performance FP32: ~60 TFLOPS with H100.

Evolution of NVIDIA GPUs

Architectures

Ampere (2020-2021)

- Significant AI performance increase with 3rd generation Tensor Cores.
- FP64 performance doubled compared to Volta.
- Max performance FP32: ~19.5 TFLOPS with A100.

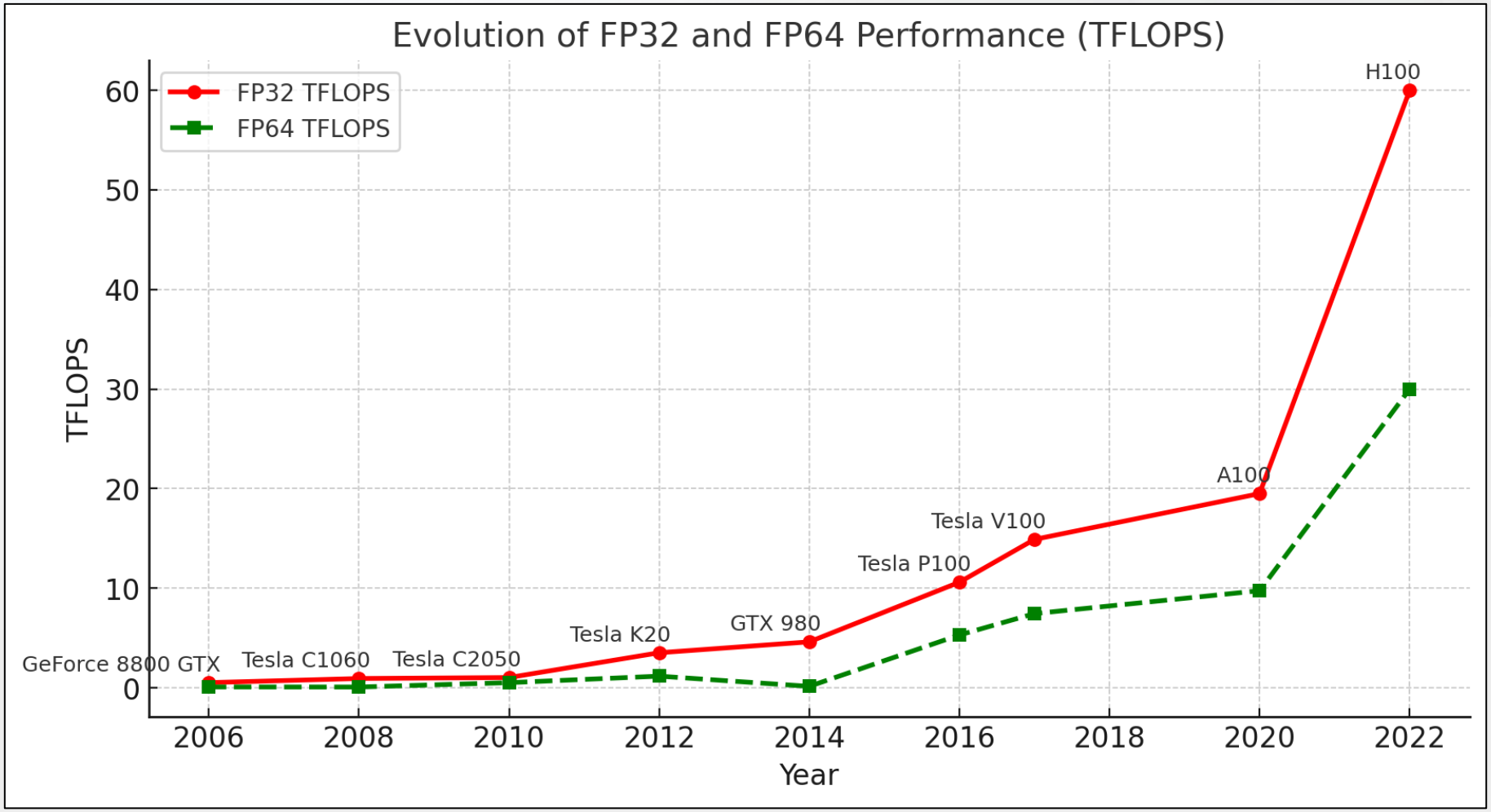
(Grace) **Hopper (2022-2023)**

- Transition to 4 nm manufacturing process (80 billion transistors)
  - Better energy efficiency.
- Support for FP8 for AI → 60 PetaFLOPS on H100!
- Max performance FP32: ~60 TFLOPS with H100.

Ada Lovelace architecture for gaming

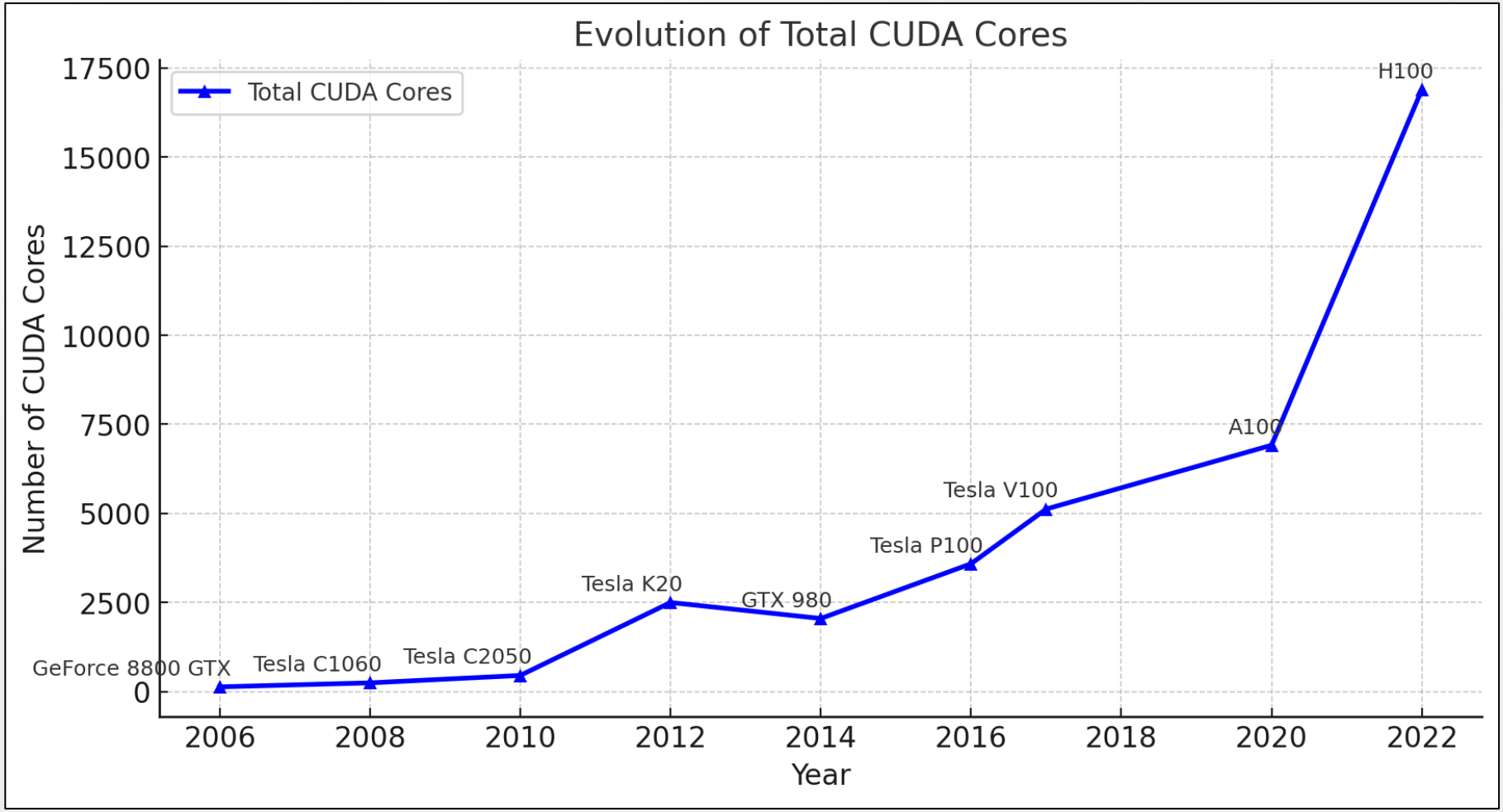
Evolution of NVIDIA GPUs

Architectures



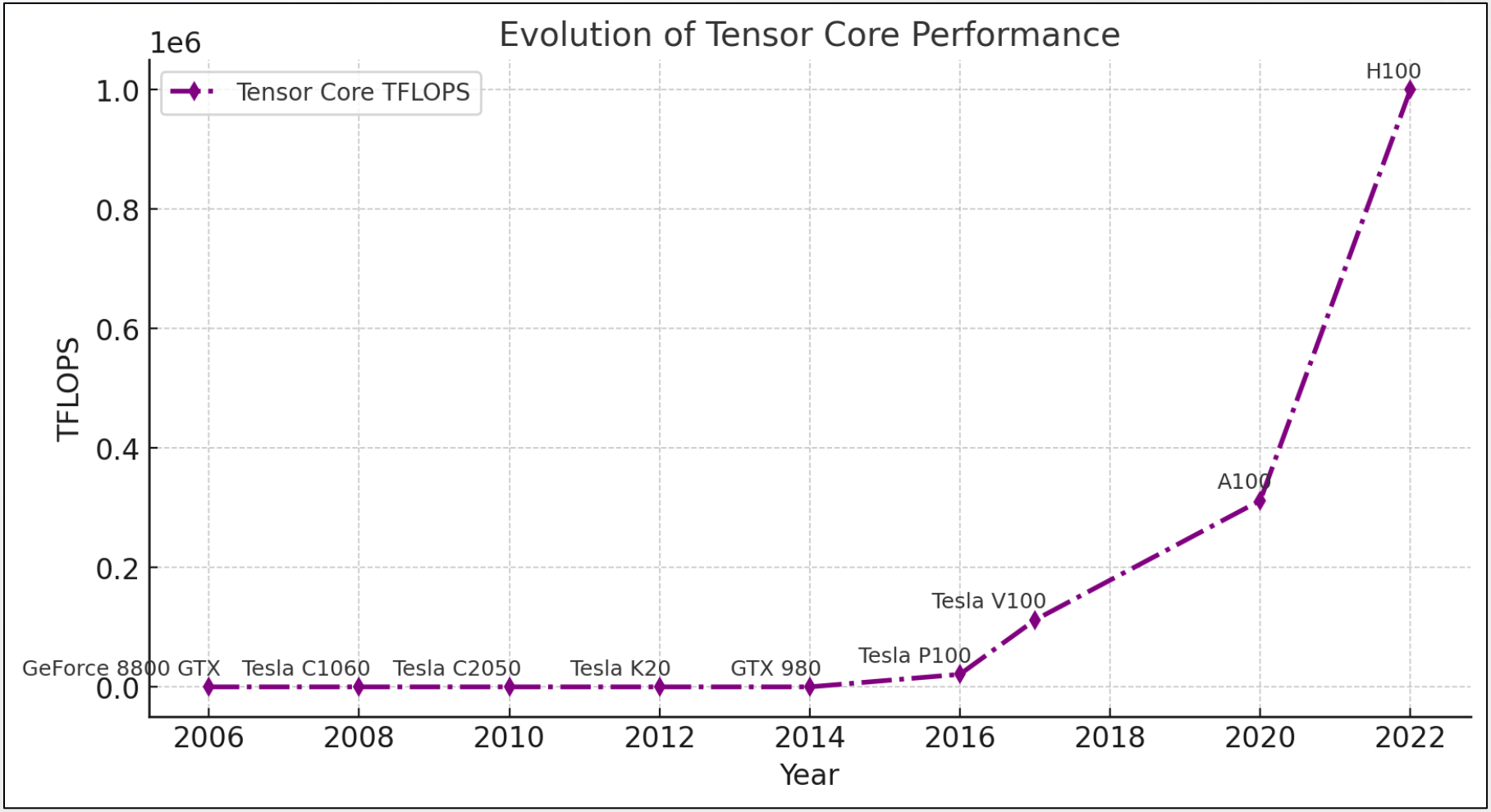
Evolution of NVIDIA GPUs

Architectures



Evolution of NVIDIA GPUs

Architectures



Evolution of NVIDIA GPUs

Architectures

Ampere (2020-2021)

- Significant AI performance increase with 3rd generation Tensor Cores.
- FP64 performance doubled compared to Volta.
- Max performance FP32: ~19.5 TFLOPS with A100.

(Grace) **Hopper (2022-2023)**

- Transition to 4 nm manufacturing process (80 billion transistors)
  - Better energy efficiency.
- Support for FP8 for AI → 60 PetaFLOPS on H100!
- Max performance FP32: ~60 TFLOPS with H100.

Grace CPU

NVIDIA's Arm-based CPU for supercomputers and data centers, designed to provide high bandwidth, high efficiency, and very tight coupling with NVIDIA GPUs.

- It is less "general-purpose" and much more oriented toward next-generation accelerated systems.

Evolution of NVIDIA GPUs

Architectures

Ampere (2020-2021)

- Significant AI performance increase with 3rd generation Tensor Cores.
- FP64 performance doubled compared to Volta.
- Max performance FP32: ~19.5 TFLOPS with A100.

(Grace) **Hopper (2022-2023)**

- Transition to 4 nm manufacturing process (80 billion transistors)
  - Better energy efficiency.
- Support for FP8 for AI → 60 PetaFLOPS on H100!
- Max performance FP32: ~60 TFLOPS with H100.

Grace CPU Superchip

A high-performance processor designed for **artificial intelligence (AI)** and **high-performance computing (HPC)** applications.

- It is a special configuration with two Grace CPU connected coherently through **NVLink-C2C** at **900 GB/s**, forming a module with **144 cores**, up to **960 GB of LPDDR5X memory with ECC**, and up to **1 TB/s of memory bandwidth**.

Evolution of NVIDIA GPUs

Architectures

Ampere (2020-2021)

- Significant AI performance increase with 3rd generation Tensor Cores.
- FP64 performance doubled compared to Volta.
- Max performance FP32: ~19.5 TFLOPS with A100.

(Grace) **Hopper (2022-2023)**

- Transition to 4 nm manufacturing process (80 billion transistors)
  - Better energy efficiency.
- Support for FP8 for AI → 60 PetaFLOPS on H100!
- Max performance FP32: ~60 TFLOPS with H100.

AI SUPERCHIP

Grace Hopper Superchip

A hybrid (on chip) solution combining the **Grace CPU** with the **Hopper GPU architecture,** aimed at enhancing performance in AI and HPC (with very tight integration)

Evolution of NVIDIA GPUs
Architectures

Blackwell (2024)

- 5th generation Tensor Cores.
- Custom 4 nm manifacturing process (208 billion transistors)
  - Higher energy efficiency
- Introduces support for lower-precision formats (FP4 and FP6)
- Decompression Engine to handle Database queries
  - highest performance in data analytics and data science
- Max performance FP32:
  - with GeForce RTX 5090 (for graphics) **~100** TFLOPS
  - with GB200 NVL72 – a rack-scale system designed to deliver exceptional performance for AI and high-performance computing (HPC)
    - 6 Grace CPUs and 72 Blackwell GPUs
    - ~180 TFLOPS per tray,
    - **~6480** TFLOPS per rack (6.48 PFLOPS).

Evolution of NVIDIA GPUs
Architectures

The Grace CPU era – 1/2

Designed mainly for **HPC, AI, and data centers**, not for desktop PCs, the Grace CPU is important because, after years of focusing primarily on GPUs, NVIDIA developed **its own high-performance CPU** to build systems in which CPU and GPU can work together much more closely than in traditional architectures.
It's a CPU with

- Many cores (144)
- high memory bandwidth, useful for scientific and AI workloads that are strongly memory-bound;
- good energy efficiency;
- strong data movement capabilities;
- very tight integration with NVIDIA GPUs: it was created to prevent the CPU from becoming the bottleneck for the GPU in a modern accelerated node. In a traditional system, CPU and GPU communicate through PCIe, which has higher latency and lower bandwidth than the integration NVIDIA achieves with Grace.

It is available either as a standalone **CPU-only** system (the Grace CPU Superchip) or as part of a **heterogeneous on-chip system** (usually called **AI Superchip**), where it is tightly coupled with an NVIDIA GPU, such as Hopper or, more recently, Blackwell.
Now NVIDIA can produce **its own HPC systems**, mainly oriented to scientific AI, typically deployed within a Linux-based software environment.

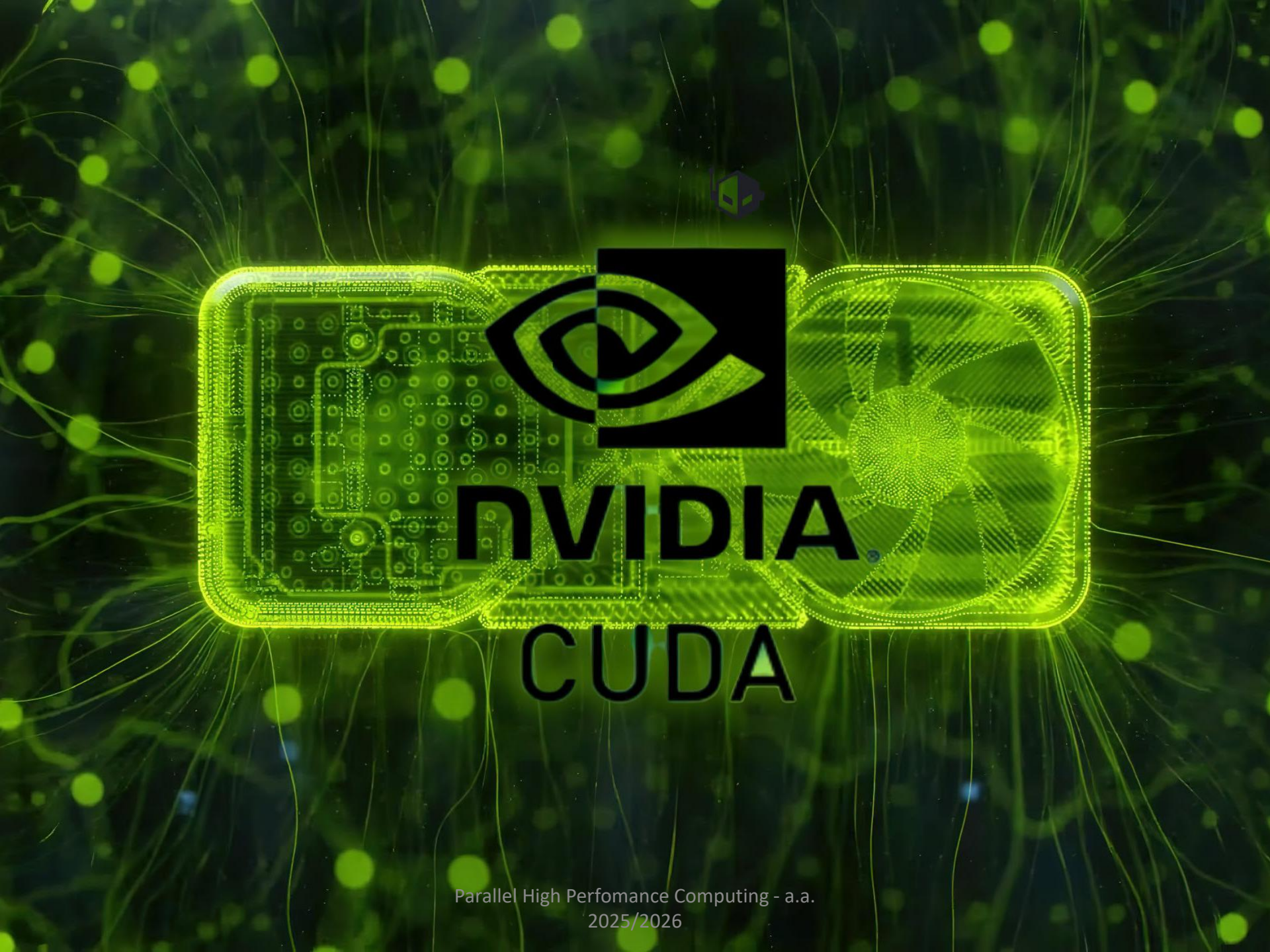
Evolution of NVIDIA GPUs
Architectures

The Grace CPU era – 2/2

In **NVIDIA-integrated platforms**, the operating system is called DGX OS, which is a customized and fully validated Ubuntu-based distribution including system-specific optimizations, NVIDIA drivers, monitoring tools, and the broader AI software stack.
In general, Grace Hopper platforms are supported on major enterprise Linux distributions such as Ubuntu, Red Hat Enterprise Linux, and SUSE Linux Enterprise Server, reflecting NVIDIA's strategy of building these architectures on a standard Linux ecosystem while tightly integrating the hardware and software layers.

Anyway, in a full Nvidia system (instead of a standard Ubuntu workstation with a GPU) the operating system (typically DGX OS) is not merely installed on top of the hardware as a generic software layer, it is co-developed and validated together with the platform itself, so that the Grace CPU, the GPU, coherent memory, drivers, runtime libraries, and system tools operate as a unified and optimized stack rather than as loosely assembled components.

NVIDIA has progressively extended the Grace Blackwell family beyond data-center and rack-scale AI infrastructure **toward desktop**, personal AI, and developer workstation systems: these desktop-scale machines are intended for local development, prototyping, fine-tuning, and inference.


NVIDIA
CUDA

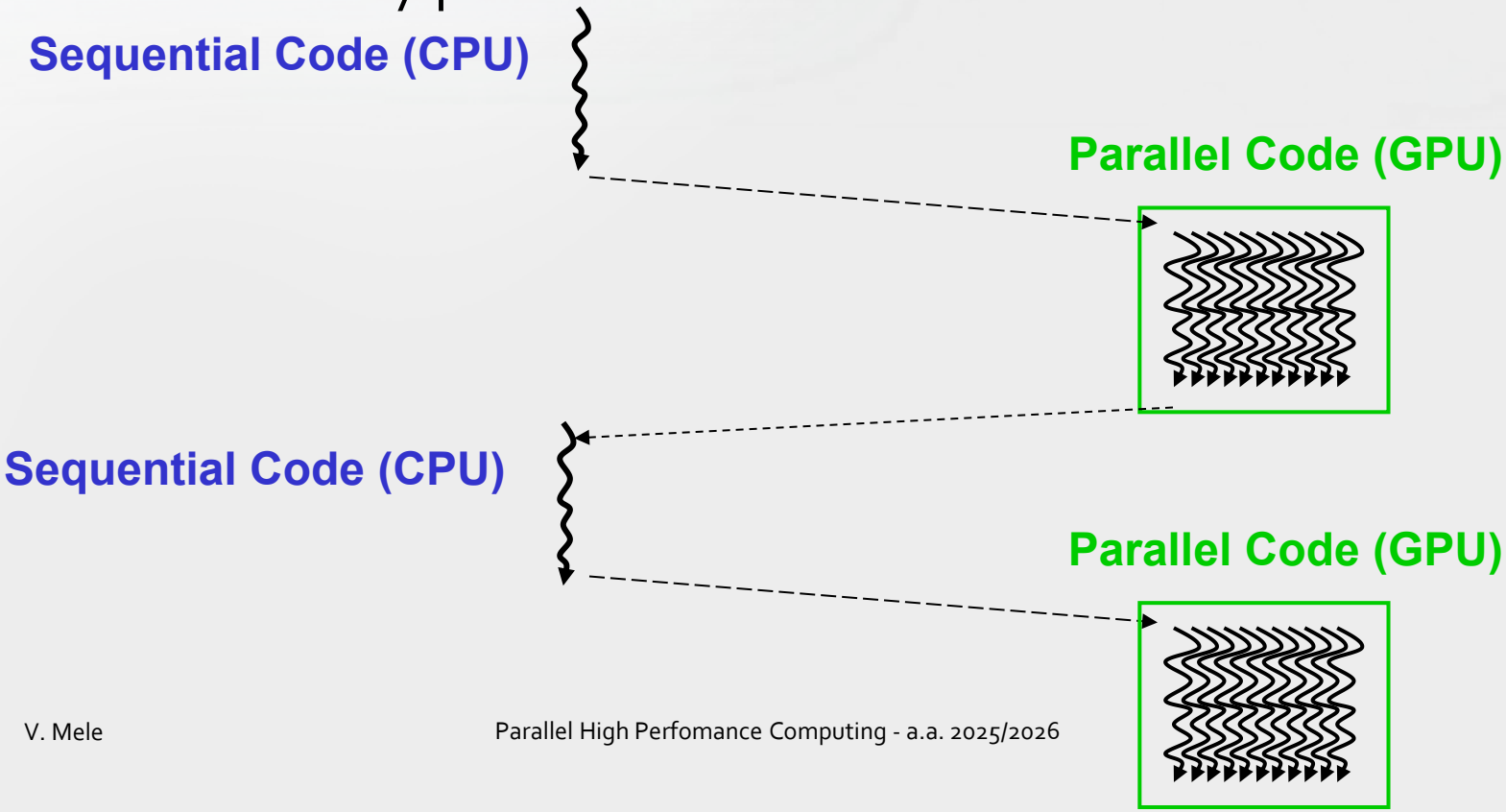
CPU-GPU System

- CPU and GPU access separate memory
- Explicit allocation and transfer between the two memories are required
- Managed by the host
  - Device memory allocation
  - Transfer to and from the device memory
  - Available APIs

CPU-GPU System

A Program on CPU+GPU:

- Serial or low-parallel sections in the CPU code
- Massively parallel sections in the GPU's SPMD code



CUDA

Differences between GPU and CPU threads:

- GPU threads are extremely lightweight
- Much lower overhead for creation
- GPUs require thousands of threads for full efficiency
- Multi-core CPUs require only a few threads

Definitions

- Device = GPU
- Host = CPU
- Kernel = A function called by the host and executed on a device

CUDA

- The GPU is composed of Streaming Multiprocessors (SMs).
- Each SM has a number (64 on V100) CUDA cores and an amount of shared memory (96 KB on V100).
- Threads are executed in warps (of 32 threads on V100).

CUDA

The **CUDA architecture** is organized into hierarchical levels.

Hardware:

- **GPU** → Composed of **Streaming Multiprocessors (SMs)**
- **Streaming Multiprocessor (SM)** → Executes multiple **Thread Blocks**
- **Thread Block** → Contains multiple **Threads**
- **Thread** → Executes instructions in parallel on an **ALU (Arithmetic Logic Unit)**

Parallel execution model:

- **Grid:** A set of Thread Blocks
- **Thread Blocks:** A set of Threads
- **Thread:** The smallest execution unit

CUDA

The **CUDA architecture** is organized into hierarchical levels.

CUDA-C Structure	CUDA Hardware Mapping
Grid (gridDim)	**GPU**
Block (blockDim)	**Streaming Multiprocessor (SM)**
Thread (threadIdx)	**CUDA Core (ALU Unit)**

Grid → Executes across the entire GPU.
Blocks → Assigned to SMs.
Threads → Executed on CUDA cores in warps (of 32 threads).

CUDA

The **CUDA architecture** is organized into hierarchical levels.

Memory Type	Access	Latency	Usage
Global Memory	Accessible by all threads	High (~400-600 clock cycles)	Used for large datasets
Shared Memory	Shared among threads in a block	Low (~30 clock cycles)	Used for data exchange within a block
Registers	Accessible only by the thread	Very low (~1 clock cycle)	For private thread variables

Shared Memory → Used to accelerate data access.

CUDA

- A **grid** consists of multiple **thread blocks**.
  - They can be 1D, 2D, 3D
- Blocks are executed **in parallel** on **SMs**.
  - They can be 1D, 2D, 3D
  - All the blocks in a grid have the same structure
- This organization simplifies memory referencing when working with multidimensional data
  - Image processing
  - Solving PDEs on volumes

CUDA

CUDA Concept	Hardware Architecture	CUDA-C Example
Grid	GPU (a collection of multiprocessors)	<<<numBlocks, numThreads>>>
Block	Multiprocessor (Streaming Multiprocessor, SM)	blockIdx.x, blockDim.x
Thread	CUDA core inside an SM	threadIdx.x, threadIdx.y
Warp (32 threads)	Minimum execution unit (SIMD)	Executed together in the SM
Shared Memory	Fast memory inside each SM	__shared__ float data[];
Global Memory	Memory accessible by the entire GPU	cudaMalloc(), cudaMemcpy()

CUDA

On an NVIDIA V100:
- 80 SMs (Streaming Multiprocessors).
- 5120 CUDA cores.
- maximum of 2048 active threads per SM

- Launching a CUDA Kernel

```
kernelname <<<numBlocks, threadsPerBlock>>>(actual parameters);
```

- This launches the CUDA kernel on the GPU.
  - **numBlocks** → Number of thread blocks (mapped to SMs).
  - **threadsPerBlock** → Number of threads per block (mapped to CUDA cores inside each SM).
- Each thread has a unique ID within the block and grid.

All structured variables with three fields x,y,z

Variable	Description
threadIdx	Identifies a **thread inside a block**.
blockIdx	Identifies a **block inside the grid**.
blockDim	Number of **threads per block**.
gridDim	Number of **blocks per grid**.

CUDA

On an NVIDIA V100:
- 80 SMs (Streaming Multiprocessors).
- 5120 CUDA cores.
- maximum of 2048 active threads per SM

- Launching a CUDA Kernel

```
kernelname <<<numBlocks, threadsPerBlock>>>(actual parameters);
```

- This launches the CUDA kernel on the GPU.
  - **numBlocks** → Number of thread blocks (mapped to SMs) → gridDim.
  - **threadsPerBlock** → Number of threads per block (mapped to CUDA cores inside each SM). → blockDim
- Each thread has a unique ID within the block and grid.
- If they are 1D structures
  - **blockIdx.x**
  - **threadIdx.x**
  - **blockDim.x**
  - **gridDim.x**

```
int global_idx_x = blockIdx.x * blockDim.x + threadIdx.x;
```

CUDA

On an NVIDIA V100:
- 80 SMs (Streaming Multiprocessors).
- 5120 CUDA cores.
- maximum of 2048 active threads per SM

- Launching a CUDA Kernel

```
kernelname <<<numBlocks, threadsPerBlock>>>(actual parameters);
```

- This launches the CUDA kernel on the GPU.
  - **numBlocks** → Number of thread blocks (mapped to SMs) → gridDim.
  - **threadsPerBlock** → Number of threads per block (mapped to CUDA cores inside each SM). → blockDim
- Each thread has a unique ID within the block and grid.
- If they are 1D structures
  - **blockIdx.x**
  - **threadIdx.x**
  - **blockDim.x**
  - **gridDim.x**

Example
- **Grid:** 4 blocks
- **Each Block:** 256 threads
- **Total threads:** 4 × 256 = 1024

```
int global_idx_x = blockIdx.x * blockDim.x + threadIdx.x;
```

CUDA

On an NVIDIA V100:
- 80 SMs (Streaming Multiprocessors).
- 5120 CUDA cores.
- maximum of 2048 active threads per SM

- Launching a CUDA Kernel

```
kernelname <<<numBlocks, threadsPerBlock>>>(actual parameters);
```

- This launches the CUDA kernel on the GPU.
  - **numBlocks** → Number of thread blocks (mapped to SMs) → gridDim.
  - **threadsPerBlock** → Number of threads per block (mapped to CUDA cores inside each SM). → blockDim
- Each thread has a unique ID within the block and grid.
- If they are 1D structures
  - **blockIdx.x**
  - **threadIdx.x**
  - **blockDim.x**
  - **gridDim.x**

global index to access the correct memory location

```
int global_idx_x = blockIdx.x * blockDim.x + threadIdx.x;
```

CUDA

- GPU memory allocation and deallocation are handled explicitly using `cudaMalloc()` and `cudaFree()`.
  - These functions are equivalent to malloc() and free() in standard C but are used for device (GPU) memory.

Memory allocation

```
cudaError_t cudaMalloc(void **devPtr, size_t size);
```

- devPtr → Pointer to the allocated memory on the GPU.
- size → Number of bytes to allocate.
- Returns a CUDA error code (use cudaSuccess to check if it succeeded).

CUDA

- GPU memory allocation and deallocation are handled explicitly using `cudaMalloc()` and `cudaFree()`.
  - These functions are equivalent to malloc() and free() in standard C but are used for device (GPU) memory.

Memory allocation

Example:

```
float *d_A;
cudaMalloc((void**)&d_A, 1024 * sizeof(float));
                // Allocates memory for 1024 floats
```

- d_A is a pointer to GPU memory.
- 1024 * sizeof(float) reserves space for 1024 floating-point numbers in global memory.

CUDA

- GPU memory allocation and deallocation are handled explicitly using `cudaMalloc()` and `cudaFree()`.
  - These functions are equivalent to malloc() and free() in standard C but are used for device (GPU) memory.

Memory deallocation

```
cudaError_t cudaFree(void *devPtr);
```

  - devPtr → Pointer to the GPU memory to be deallocated.

CUDA

- GPU memory allocation and deallocation are handled explicitly using `cudaMalloc()` and `cudaFree()`.
  - These functions are equivalent to malloc() and free() in standard C but are used for device (GPU) memory.

Memory deallocation

Example:

```
cudaFree(d_A);
```

- d_A is a pointer to GPU memory.

CUDA

```
#include <cuda_runtime.h>
#include <stdio.h>

int main() {
    float *d_A;
    size_t size = 1024 * sizeof(float);

    // Allocate memory on GPU
    if (cudaMalloc((void**)&d_A, size) != cudaSuccess) {
        fprintf(stderr, "cudaMalloc failed!\n");
        return -1;
    }

    printf("Memory allocated on GPU!\n");

    // Free GPU memory
    cudaFree(d_A);
    printf("Memory freed!\n");

    return 0;
}
```

CUDA

- CUDA operates in a heterogeneous memory model, where the CPU (host) and GPU (device) have separate memory spaces.
- cudaMemcpy() is used to transfer data between the host (CPU) and device (GPU).

Memory transfer

```
cudaError_t cudaMemcpy(void *dst, const void *src, size_t count,
                       cudaMemcpyKind kind);
```

- dst → Destination pointer (where the data is copied).
- src → Source pointer (where the data is copied from).
- count → Number of bytes to copy.
- kind → Direction of memory transfer (cudaMemcpyKind).

CUDA

- CUDA operates in a heterogeneous memory model, where the CPU (host) and GPU (device) have separate memory spaces.
- cudaMemcpy() is used to transfer data between the host (CPU) and device (GPU).

Memory transfer

```
cudaError_t cudaMemcpy(void *dst, const void *src, size_t count,
                       cudaMemcpyKind kind);
```

- dst → D
- src → S
- count →
- kind → k

cudaMemcpyKind

Value	Description
cudaMemcpyHostToDevice	Copy from **host (CPU) → device (GPU)**
cudaMemcpyDeviceToHost	Copy from **device (GPU) → host (CPU)**
cudaMemcpyDeviceToDevice	Copy **within device memory (GPU → GPU)**
cudaMemcpyHostToHost	Copy **within host memory (CPU → CPU)**

CUDA

```
#include <cuda_runtime.h>
#include <stdio.h>

int main() {
    int N = 1024;
    size_t size = N * sizeof(float);

    // Allocate memory on host
    float *h_A = (float*)malloc(size);
    float *h_B = (float*)malloc(size);

    // Allocate memory on device
    float *d_A;
    cudaMalloc((void**)&d_A, size);

    // Initialize host data
    for (int i = 0; i < N; i++) {
        h_A[i] = i * 1.0f;
    }
```

```
    // Copy from Host to Device
    cudaMemcpy(d_A, h_A, size,
               cudaMemcpyHostToDevice);

    // Copy back from Device to Host
    cudaMemcpy(h_B, d_A, size,
               cudaMemcpyDeviceToHost);

    // Free memory
    free(h_A);
    free(h_B);
    cudaFree(d_A);

    printf("Memory transfer completed
             successfully!\n");
    return 0;
}
```

CUDA

```
#include <cuda_runtime.h>
#include <stdio.h>

int main() {
    int N = 1024;
    size_t size = N * sizeof(float);

    // Allocate memory on host
    float *h_A = (float*)malloc(size);
    float *h_B = (float*)malloc(size);

    // Allocate memory on device
    float *d_A;
    cudaMalloc((void**)&d_A, size);

    // Initialize host data
    for (int i = 0; i < N; i++) {
        h_A[i] = i * 1.0f;
    }
```

```
    // Copy from Host to Device
    cudaMemcpy(d_A, h_A, size,
               cudaMemcpyHostToDevice);

    // Copy back from Device to Host
    cudaMemcpy(h_B, d_A, size,
               cudaMemcpyDeviceToHost);

    // Free memory
    free(h_A);
    free(h_B);
    cudaFree(d_A);

    printf("Memory transfer completed
             successfully!\n");
    return 0;
}
```

CUDA

```
#include <cuda_runtime.h>
#include <stdio.h>

int main() {
    int N = 1024;
    size_t size = N * sizeof(float);

    // Allocate memory on host
    float *h_A = (float*)malloc(size);
    float *h_B = (float*)malloc(size);

    // Allocate memory on device
    float *d_A;
    cudaMalloc((void**)&d_A, size);

    // Initialize host data
    for (int i = 0; i < N; i++) {
        h_A[i] = i * 1.0f;
    }
```

```
    // Copy from Host to Device
    cudaMemcpy(d_A, h_A, size,
               cudaMemcpyHostToDevice);

    // Copy back from Device to Host
    cudaMemcpy(h_B, d_A, size,
               cudaMemcpyDeviceToHost);

    // Free memory
    free(h_A);
    free(h_B);
    cudaFree(d_A);

    printf("Memory transfer completed
             successfully!\n");
    return 0;
}
```

CUDA

```
#include <cuda_runtime.h>
#include <stdio.h>

int main() {
    int N = 1024;
    size_t size = N * sizeof(float);

    // Allocate memory on host
    float *h_A = (float*)malloc(size);
    float *h_B = (float*)malloc(size);

    // Allocate memory on device
    float *d_A;
    cudaMalloc((void**)&d_A, size);

    // Initialize host data
    for (int i = 0; i < N; i++) {
        h_A[i] = i * 1.0f;
    }
```

```
    // Copy from Host to Device
    cudaMemcpy(d_A, h_A, size,
               cudaMemcpyHostToDevice);

    // Copy back from Device to Host
    cudaMemcpy(h_B, d_A, size,
               cudaMemcpyDeviceToHost);

    // Free memory
    free(h_A);
    free(h_B);
    cudaFree(d_A);

    printf("Memory transfer completed
             successfully!\n");
    return 0;
}
```

CUDA

- cudaMemcpy() is blocking → The CPU waits for the transfer to complete before executing the next instruction.

- Always match memory allocation and transfer directions:
  - Host (malloc()) ↔ Device (cudaMalloc()).
  - Use cudaMemcpyHostToDevice and cudaMemcpyDeviceToHost accordingly.

- For large data transfers, consider using **cudaMemcpyAsync()** for overlapping computation and memory transfer.

- Efficient memory transfer is crucial for high-performance CUDA applications!

Installing CUDA on the local machine

- Go to this URL and follow the on-screen instructions:

https://developer.nvidia.com/cuda-downloads

- All CUDA documentation is available at this URL

https://docs.nvidia.com/cuda/

Installing CUDA on the local machine

Verify that your **GPU supports CUDA**

- Go to this URL and follow the on-screen instructions:

https://developer.nvidia.com/cuda-downloads

- All CUDA documentation is available at this URL

https://docs.nvidia.com/cuda/